

11.5 Minimum Spanning Trees

11.5.1 Introduction

Links ➤ A company plans to build a communications network connecting its five computer centers. Any pair of these centers can be linked with a leased telephone line. Which links should be made to ensure that there is a path between any two computer centers so that the total cost of the network is minimized? We can model this problem using the weighted graph shown in Figure 1, where vertices represent computer centers, edges represent possible leased lines, and the weights on edges are the monthly lease rates of the lines represented by the edges. We can solve this problem by finding a spanning tree so that the sum of the weights of the edges of the tree is minimized. Such a spanning tree is called a **minimum spanning tree**.

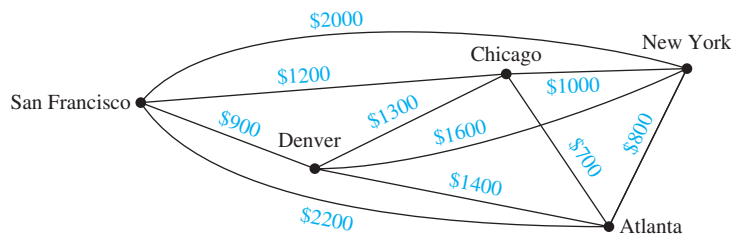


FIGURE 1 A weighted graph showing monthly lease costs for lines in a computer network.

11.5.2 Algorithms for Minimum Spanning Trees

A wide variety of problems are solved by finding a spanning tree in a weighted graph such that the sum of the weights of the edges in the tree is a minimum.

Definition 1

A *minimum spanning tree* in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.

Demo

We will present two algorithms for constructing minimum spanning trees. Both proceed by successively adding edges of smallest weight from those edges with a specified property that have not already been used. Both are greedy algorithms. Recall from Section 3.1 that a greedy algorithm is a procedure that makes an optimal choice at each of its steps. Optimizing at each step does not guarantee that the optimal overall solution is produced. However, the two algorithms presented in this section for constructing minimum spanning trees are greedy algorithms that do produce optimal solutions.

Links

The first algorithm that we will discuss was originally discovered by the Czech mathematician Vojtěch Jarník in 1930, who described it in a paper in an obscure Czech journal. The algorithm became well known when it was rediscovered in 1957 by Robert Prim. Because of this, it is known as **Prim's algorithm** (and sometimes as the **Prim-Jarník algorithm**). Begin by choosing any edge with smallest weight, putting it into the spanning tree. Successively add to the tree edges of minimum weight that are incident to a vertex already in the tree, never forming a simple circuit with those edges already in the tree. Stop when $n - 1$ edges have been added.

Later in this section, we will prove that this algorithm produces a minimum spanning tree for any connected weighted graph. Algorithm 1 gives a pseudocode description of Prim's algorithm.

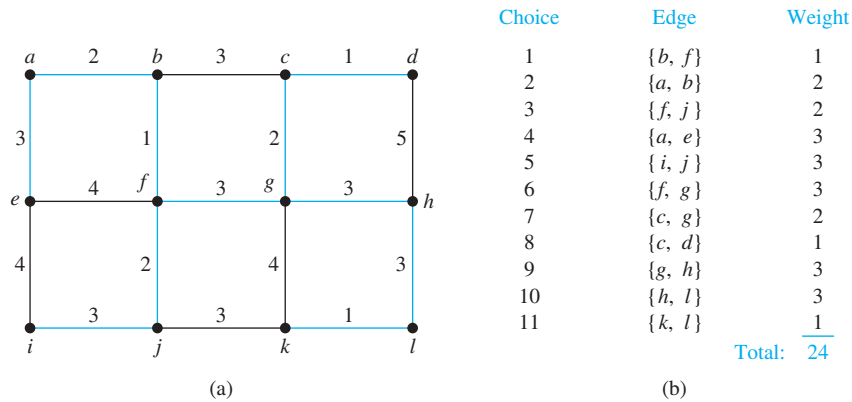


FIGURE 4 A minimum spanning tree produced using Prim's algorithm.

Links

The second algorithm we will discuss was discovered by Joseph Kruskal in 1956, although the basic ideas it uses were described much earlier. To carry out **Kruskal's algorithm**, choose an edge in the graph with minimum weight.

Successively add edges with minimum weight that do not form a simple circuit with those edges already chosen. Stop after $n - 1$ edges have been selected.

The proof that Kruskal's algorithm produces a minimum spanning tree for every connected weighted graph is left as an exercise. Pseudocode for Kruskal's algorithm is given in Algorithm 2.

ALGORITHM 2 Kruskal's Algorithm.

```

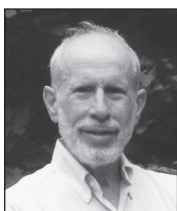
procedure Kruskal( $G$ : weighted connected undirected graph with  $n$  vertices)
 $T :=$  empty graph
for  $i := 1$  to  $n - 1$ 
     $e :=$  any edge in  $G$  with smallest weight that does not form a simple circuit
        when added to  $T$ 
     $T := T$  with  $e$  added
return  $T$  { $T$  is a minimum spanning tree of  $G$ }
    
```

Links



Courtesy of Robert Clay Prim

ROBERT CLAY PRIM (BORN 1921) Robert Prim, born in Sweetwater, Texas, received his B.S. in electrical engineering in 1941 and his Ph.D. in mathematics from Princeton University in 1949. He was an engineer at the General Electric Company from 1941 until 1944, an engineer and mathematician at the United States Naval Ordnance Lab from 1944 until 1949, and a research associate at Princeton University from 1948 until 1949. Among the other positions he has held are director of mathematics and mechanics research at Bell Telephone Laboratories from 1958 until 1961 and vice president of research at Sandia Corporation. He is currently retired.



Courtesy of Joseph Kruskal

JOSEPH BERNARD KRUSKAL (1928–2010) Joseph Kruskal was born in New York City, where his father was a fur dealer and his mother promoted the art of origami on early television. Kruskal attended the University of Chicago and received his Ph.D. from Princeton University in 1954. He was an instructor in mathematics at Princeton and at the University of Wisconsin, and later he was an assistant professor at the University of Michigan. In 1959 he became a member of the technical staff at Bell Laboratories, where he worked until his retirement in the late 1990s. Kruskal discovered his algorithm for producing minimum spanning trees when he was a second-year graduate student. He was not sure his $2\frac{1}{2}$ -page paper on this subject was worthy of publication, but was convinced by others to submit it. His research interests included statistical linguistics and psychometrics. Besides his work on minimum spanning trees, Kruskal is also known for contributions to multidimensional scaling. It is noteworthy that Joseph Kruskal's two brothers, Martin and William, also were well known mathematicians.

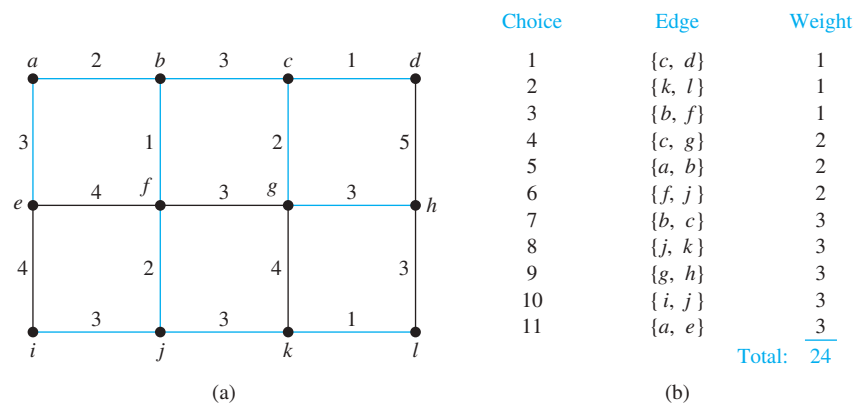


FIGURE 5 A minimum spanning tree produced by Kruskal’s algorithm.

The reader should note the difference between Prim’s and Kruskal’s algorithms. In Prim’s algorithm edges of minimum weight that are incident to a vertex already in the tree, and not forming a circuit, are chosen; whereas in Kruskal’s algorithm edges of minimum weight that are not necessarily incident to a vertex already in the tree, and that do not form a circuit, are chosen. Note that as in Prim’s algorithm, if the edges are not ordered, there may be more than one choice for the edge to add at a stage of this procedure. Consequently, the edges need to be ordered for the procedure to be deterministic. Example 3 illustrates how Kruskal’s algorithm is used.

EXAMPLE 3 Use Kruskal’s algorithm to find a minimum spanning tree in the weighted graph shown in Figure 3.

Extra Examples

Solution: A minimum spanning tree and the choices of edges at each stage of Kruskal’s algorithm are shown in Figure 5.

We will now prove that Prim’s algorithm produces a minimum spanning tree of a connected weighted graph.



Proof: Let G be a connected weighted graph. Suppose that the successive edges chosen by Prim’s algorithm are $e_1, e_2, \dots, e_{n-1}$. Let S be the tree with $e_1, e_2, \dots, e_{n-1}$ as its edges, and let S_k be the tree with $e_1, e_2, \dots, e_k$ as its edges. Let T be a minimum spanning tree of G containing the edges $e_1, e_2, \dots, e_k$, where k is the maximum integer with the property that a minimum spanning tree exists containing the first k edges chosen by Prim’s algorithm. The theorem follows if we can show that $S = T$.

Suppose that $S \neq T$, so that $k < n - 1$. Consequently, T contains $e_1, e_2, \dots, e_k$, but not e_{k+1} . Consider the graph made up of T together with e_{k+1} . Because this graph is connected and has n edges, too many edges to be a tree, it must contain a simple circuit. This simple circuit must contain e_{k+1} because there was no simple circuit in T . Furthermore, there must be an

Links

HISTORICAL NOTE Joseph Kruskal and Robert Prim developed their algorithms for constructing minimum spanning trees in the mid-1950s. However, they were not the first people to discover such algorithms. For example, the work of the anthropologist Jan Czekanowski, in 1909, contains many of the ideas required to find minimum spanning trees. In 1926, Otakar Boruvka described methods for constructing minimum spanning trees in work relating to the construction of electric power networks, and as mentioned in the text what is now called Prim’s algorithm was discovered by Vojtěch Jarník in 1930.

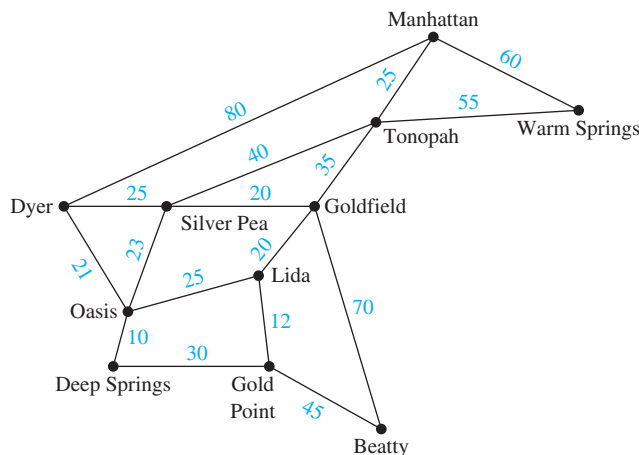
edge in the simple circuit that does not belong to S_{k+1} because S_{k+1} is a tree. By starting at an endpoint of e_{k+1} that is also an endpoint of one of the edges $e_1, \dots, e_k$, and following the circuit until it reaches an edge not in S_{k+1} , we can find an edge e not in S_{k+1} that has an endpoint that is also an endpoint of one of the edges $e_1, e_2, \dots, e_k$.

By deleting e from T and adding e_{k+1} , we obtain a tree T' with $n - 1$ edges (it is a tree because it has no simple circuits). Note that the tree T' contains $e_1, e_2, \dots, e_k, e_{k+1}$. Furthermore, because e_{k+1} was chosen by Prim's algorithm at the $(k + 1)$ st step, and e was also available at that step, the weight of e_{k+1} is less than or equal to the weight of e . From this observation, it follows that T' is also a minimum spanning tree, because the sum of the weights of its edges does not exceed the sum of the weights of the edges of T . This contradicts the choice of k as the maximum integer such that a minimum spanning tree exists containing $e_1, \dots, e_k$. Hence, $k = n - 1$, and $S = T$. It follows that Prim's algorithm produces a minimum spanning tree. $\triangleleft$

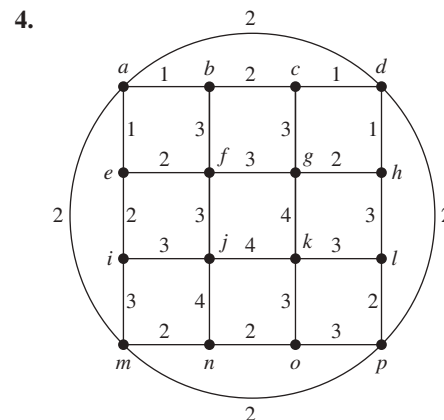
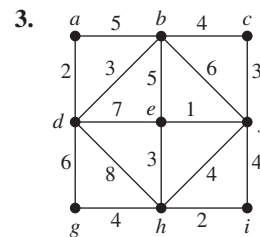
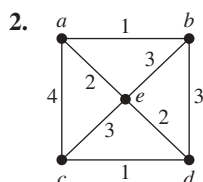
It can be shown (see [CoLeRiSt09]) that to find a minimum spanning tree of a graph with m edges and n vertices, Kruskal's algorithm can be carried out using $O(m \log m)$ operations and Prim's algorithm can be carried out using $O(m \log n)$ operations. Consequently, it is preferable to use Kruskal's algorithm for graphs that are **sparse**, that is, where m is very small compared to $C(n, 2) = n(n - 1)/2$, the total number of possible edges in an undirected graph with n vertices. Otherwise, there is little difference in the complexity of these two algorithms.

Exercises

- The roads represented by this graph are all unpaved. The lengths of the roads between pairs of towns are represented by edge weights. Which roads should be paved so that there is a path of paved roads between each pair of towns so that a minimum road length is paved? (Note: These towns are in Nevada.)



In Exercises 2–4 use Prim's algorithm to find a minimum spanning tree for the given weighted graph.



- Use Kruskal's algorithm to design the communications network described at the beginning of the section.
- Use Kruskal's algorithm to find a minimum spanning tree for the weighted graph in Exercise 2.
- Use Kruskal's algorithm to find a minimum spanning tree for the weighted graph in Exercise 3.
- Use Kruskal's algorithm to find a minimum spanning tree for the weighted graph in Exercise 4.

9. Find a connected weighted simple graph with the fewest edges possible that has more than one minimum spanning tree.
10. A **minimum spanning forest** in a weighted graph is a spanning forest with minimal weight. Explain how Prim's and Kruskal's algorithms can be adapted to construct minimum spanning forests.

A **maximum spanning tree** of a connected weighted undirected graph is a spanning tree with the largest possible weight.

11. Devise an algorithm similar to Prim's algorithm for constructing a maximum spanning tree of a connected weighted graph.
12. Devise an algorithm similar to Kruskal's algorithm for constructing a maximum spanning tree of a connected weighted graph.
13. Find a maximum spanning tree for the weighted graph in Exercise 2.
14. Find a maximum spanning tree for the weighted graph in Exercise 3.
15. Find a maximum spanning tree for the weighted graph in Exercise 4.
16. Find the second least expensive communications network connecting the five computer centers in the problem posed at the beginning of the section.
- *17. Devise an algorithm for finding the second shortest spanning tree in a connected weighted graph.
- *18. Show that an edge with smallest weight in a connected weighted graph must be part of any minimum spanning tree.
19. Show that there is a unique minimum spanning tree in a connected weighted graph if the weights of the edges are all different.
20. Suppose that the computer network connecting the cities in Figure 1 must contain a direct link between New York and Denver. What other links should be included so that there is a link between every two computer centers and the cost is minimized?
21. Find a spanning tree with minimal total weight containing the edges $\{e, i\}$ and $\{g, k\}$ in the weighted graph in Figure 3.
22. Describe an algorithm for finding a spanning tree with minimal weight containing a specified set of edges in a connected weighted undirected simple graph.
23. Express the algorithm devised in Exercise 22 in pseudocode.

Sollin's algorithm produces a minimum spanning tree from a connected weighted simple graph $G = (V, E)$ by successively adding groups of edges. Suppose that the vertices in V are ordered. This produces an ordering of the edges where $\{u_0, v_0\}$ precedes $\{u_1, v_1\}$ if u_0 precedes u_1 or if $u_0 = u_1$ and v_0 precedes v_1 . The algorithm begins by simultaneously choosing the edge of least weight incident to each vertex. The first edge in the ordering is taken in the case of ties. This produces a

graph with no simple circuits, that is, a forest of trees (Exercise 24 asks for a proof of this fact). Next, simultaneously choose for each tree in the forest the shortest edge between a vertex in this tree and a vertex in a different tree. Again the first edge in the ordering is chosen in the case of ties. (This produces a graph with no simple circuits containing fewer trees than were present before this step; see Exercise 24.) Continue the process of simultaneously adding edges connecting trees until $n - 1$ edges have been chosen. At this stage a minimum spanning tree has been constructed.

- *24. Show that the addition of edges at each stage of Sollin's algorithm produces a forest.
25. Use Sollin's algorithm to produce a minimum spanning tree for the weighted graph shown in
 - a) Figure 1.
 - b) Figure 3.
- *26. Express Sollin's algorithm in pseudocode.
- **27. Prove that Sollin's algorithm produces a minimum spanning tree in a connected undirected weighted graph.
- *28. Show that the first step of Sollin's algorithm produces a forest containing at least $\lfloor n/2 \rfloor$ edges when the input is an undirected graph with n vertices.
- *29. Show that if there are r trees in the forest at some intermediate step of Sollin's algorithm, then at least $\lceil r/2 \rceil$ edges are added by the next iteration of the algorithm.
- *30. Show that when given as input an undirected graph with n vertices, no more than $\lfloor n/2^k \rfloor$ trees remain after the first step of Sollin's algorithm has been carried out and the second step of the algorithm has been carried out $k - 1$ times.
- *31. Show that Sollin's algorithm requires at most $\log n$ iterations to produce a minimum spanning tree from a connected undirected weighted graph with n vertices.
32. Prove that Kruskal's algorithm produces minimum spanning trees.
33. Show that if G is a weighted graph with distinct edge weights, then for every simple circuit of G , the edge of maximum weight in this circuit does not belong to any minimum spanning tree of G .

When Kruskal invented the algorithm that finds minimum spanning trees by adding edges in order of increasing weight as long as they do not form a simple circuit, he also invented another algorithm sometimes called the **reverse-delete algorithm**. This algorithm proceeds by successively deleting edges of maximum weight from a connected graph as long as doing so does not disconnect the graph.

34. Express the reverse-delete algorithm in pseudocode.
35. Prove that the reverse-delete algorithm always produces a minimum spanning tree when given as input a weighted graph with distinct edge weights. [Hint: Use Exercise 33.]

Key Terms and Results

TERMS

tree: a connected undirected graph with no simple circuits

forest: an undirected graph with no simple circuits

rooted tree: a directed graph with a specified vertex, called the root, such that there is a unique path to every other vertex from this root

subtree: a subgraph of a tree that is also a tree

parent of v in a rooted tree: the vertex u such that (u, v) is an edge of the rooted tree

child of a vertex v in a rooted tree: any vertex with v as its parent

sibling of a vertex v in a rooted tree: a vertex with the same parent as v

ancestor of a vertex v in a rooted tree: any vertex on the path from the root to v

descendant of a vertex v in a rooted tree: any vertex that has v as an ancestor

internal vertex: a vertex that has children

leaf: a vertex with no children

level of a vertex: the length of the path from the root to this vertex

height of a tree: the largest level of the vertices of a tree

m -ary tree: a tree with the property that every internal vertex has no more than m children

full m -ary tree: a tree with the property that every internal vertex has exactly m children

binary tree: an m -ary tree with $m = 2$ (each child may be designated as a left or a right child of its parent)

ordered tree: a tree in which the children of each internal vertex are linearly ordered

balanced tree: a tree in which every leaf is at level h or $h - 1$, where h is the height of the tree

binary search tree: a binary tree in which the vertices are labeled with items so that a label of a vertex is greater than the labels of all vertices in the left subtree of this vertex and is less than the labels of all vertices in the right subtree of this vertex

decision tree: a rooted tree where each vertex represents a possible outcome of a decision and the leaves represent the possible solutions of a problem

game tree: a rooted tree where vertices represent the possible positions of a game as it progresses and edges represent legal moves between these positions

prefix code: a code that has the property that the code of a character is never a prefix of the code of another character

minimax strategy: the strategy where the first player and second player move to positions represented by a child with maximum and minimum value, respectively

value of a vertex in a game tree: for a leaf, the payoff to the first player when the game terminates in the position represented by this leaf; for an internal vertex, the maximum or minimum of the values of its children, for an internal vertex at an even or odd level, respectively

tree traversal: a listing of the vertices of a tree

preorder traversal: a listing of the vertices of an ordered rooted tree defined recursively—the root is listed, followed by the first subtree, followed by the other in the order they occur from left to right

inorder traversal: a listing of the vertices of an ordered rooted tree defined recursively—the first subtree is listed, followed by the root, followed by the other subtrees in the order they occur from left to right

postorder traversal: a listing of the vertices of an ordered rooted tree defined recursively—the subtrees are listed in the order they occur from left to right, followed by the root

infix notation: the form of an expression (including a full set of parentheses) obtained from an inorder traversal of the binary tree representing this expression

prefix (or Polish) notation: the form of an expression obtained from a preorder traversal of the tree representing this expression

postfix (or reverse Polish) notation: the form of an expression obtained from a postorder traversal of the tree representing this expression

spanning tree: a tree containing all vertices of a graph

minimum spanning tree: a spanning tree with smallest possible sum of weights of its edges

RESULTS

A graph is a tree if and only if there is a unique simple path between every pair of its vertices.

A tree with n vertices has $n - 1$ edges.

A full m -ary tree with i internal vertices has $mi + 1$ vertices.

The relationships among the numbers of vertices, leaves, and internal vertices in a full m -ary tree (see Theorem 4 in Section 11.1)

There are at most m^h leaves in an m -ary tree of height h .

If an m -ary tree has l leaves, its height h is at least $\lceil \log_m l \rceil$. If the tree is also full and balanced, then its height is $\lceil \log_m l \rceil$.

Huffman coding: a procedure for constructing an optimal binary code for a set of symbols, given the frequencies of these symbols

depth-first search, or backtracking: a procedure for constructing a spanning tree by adding edges that form a path until this is not possible, and then moving back up the path until a vertex is found where a new path can be formed

breadth-first search: a procedure for constructing a spanning tree that successively adds all edges incident to the last set of edges added, unless a simple circuit is formed

Prim's algorithm: a procedure for producing a minimum spanning tree in a weighted graph that successively adds edges with minimal weight among all edges incident to a

vertex already in the tree so that no edge produces a simple circuit when it is added

Kruskal's algorithm: a procedure for producing a minimum spanning tree in a weighted graph that successively adds edges of least weight that are not already in the tree such that no edge produces a simple circuit when it is added

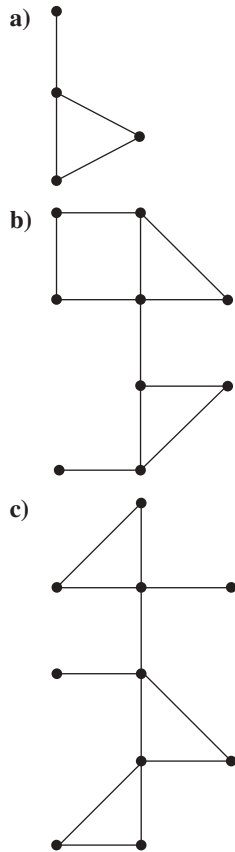
Review Questions

1. a) Define a tree. b) Define a forest.
2. Can there be two different simple paths between the vertices of a tree?
3. Give at least three examples of how trees are used in modeling.
4. a) Define a rooted tree and the root of such a tree.
b) Define the parent of a vertex and a child of a vertex in a rooted tree.
c) What are an internal vertex, a leaf, and a subtree in a rooted tree?
d) Draw a rooted tree with at least 10 vertices, where the degree of each vertex does not exceed 3. Identify the root, the parent of each vertex, the children of each vertex, the internal vertices, and the leaves.
5. a) How many edges does a tree with n vertices have?
b) What do you need to know to determine the number of edges in a forest with n vertices?
6. a) Define a full m -ary tree.
b) How many vertices does a full m -ary tree have if it has i internal vertices? How many leaves does the tree have?
7. a) What is the height of a rooted tree?
b) What is a balanced tree?
c) How many leaves can an m -ary tree of height h have?
8. a) What is a binary search tree?
b) Describe an algorithm for constructing a binary search tree.
c) Form a binary search tree for the words *vireo*, *warbler*, *egret*, *grosbeak*, *nuthatch*, and *kingfisher*.
9. a) What is a prefix code?
b) How can a prefix code be represented by a binary tree?
10. a) Define preorder, inorder, and postorder tree traversal.
b) Give an example of preorder, postorder, and inorder traversal of a binary tree of your choice with at least 12 vertices.
11. a) Explain how to use preorder, inorder, and postorder traversals to find the prefix, infix, and postfix forms of an arithmetic expression.
b) Draw the ordered rooted tree that represents $((x - 3) + ((x/4) + (x - y) \uparrow 3))$.
- c) Find the prefix and postfix forms of the expression in part (b).
12. Show that the number of comparisons used by a sorting algorithm to sort a list of n elements is at least $\lceil \log n! \rceil$.
13. a) Describe the Huffman coding algorithm for constructing an optimal code for a set of symbols, given the frequency of these symbols.
b) Use Huffman coding to find an optimal code for these symbols and frequencies: A: 0.2, B: 0.1, C: 0.3, D: 0.4.
14. Draw the game tree for nim if the starting position consists of two piles with one and four stones, respectively. Who wins the game if both players follow an optimal strategy?
15. a) What is a spanning tree of a simple graph?
b) Which simple graphs have spanning trees?
c) Describe at least two different applications that require that a spanning tree of a simple graph be found.
16. a) Describe two different algorithms for finding a spanning tree in a simple graph.
b) Illustrate how the two algorithms you described in part (a) can be used to find the spanning tree of a simple graph, using a graph of your choice with at least eight vertices and 15 edges.
17. a) Explain how backtracking can be used to determine whether a simple graph can be colored using n colors.
b) Show, with an example, how backtracking can be used to show that a graph with a chromatic number equal to 4 cannot be colored with three colors, but can be colored with four colors.
18. a) What is a minimum spanning tree of a connected weighted graph?
b) Describe at least two different applications that require that a minimum spanning tree of a connected weighted graph be found.
19. a) Describe Kruskal's algorithm and Prim's algorithm for finding minimum spanning trees.
b) Illustrate how Kruskal's algorithm and Prim's algorithm are used to find a minimum spanning tree, using a weighted graph with at least eight vertices and 15 edges.

Supplementary Exercises

- *1. Show that a simple graph is a tree if and only if it contains no simple circuits and the addition of an edge connecting two nonadjacent vertices produces a new graph that has exactly one simple circuit (where circuits that contain the same edges are not considered different).
- *2. How many nonisomorphic rooted trees are there with six vertices?
3. Show that every tree with at least one edge must have at least two pendant vertices.
4. Show that a tree with n vertices that has $n - 1$ pendant vertices must be isomorphic to $K_{1,n-1}$.
5. What is the sum of the degrees of the vertices of a tree with n vertices?
- *6. Suppose that $d_1, d_2, \dots, d_n$ are n positive integers with sum $2n - 2$. Show that there is a tree that has n vertices such that the degrees of these vertices are $d_1, d_2, \dots, d_n$.
7. Show that every tree is a planar graph.
8. Show that every tree is bipartite.
9. Show that every forest can be colored using two colors.
- A **B-tree of degree k** is a rooted tree such that all its leaves are at the same level, its root has at least two and at most k children unless it is a leaf, and every internal vertex other than the root has at least $\lceil k/2 \rceil$, but no more than k , children. Computer files can be accessed efficiently when B-trees are used to represent them.
10. Draw three different B-trees of degree 3 with height 4.
- *11. Give an upper bound and a lower bound for the number of leaves in a B-tree of degree k with height h .
- *12. Give an upper bound and a lower bound for the height of a B-tree of degree k with n leaves.
- The **binomial trees** B_i , $i = 0, 1, 2, \dots$, are ordered rooted trees defined recursively:
- Basis step:* The binomial tree B_0 is the tree with a single vertex.
- Recursive step:* Let k be a nonnegative integer. To construct the binomial tree B_{k+1} , add a copy of B_k to a second copy of B_k by adding an edge that makes the root of the first copy of B_k the leftmost child of the root of the second copy of B_k .
13. Draw B_k for $k = 0, 1, 2, 3, 4$.
14. How many vertices does B_k have? Prove that your answer is correct.
15. Find the height of B_k . Prove that your answer is correct.
16. How many vertices are there in B_k at depth j , where $0 \leq j \leq k$? Justify your answer.
17. What is the degree of the root of B_k ? Prove that your answer is correct.
18. Show that the vertex of largest degree in B_k is the root.
- A rooted tree T is called an **S_k -tree** if it satisfies this recursive definition. It is an S_0 -tree if it has one vertex. For $k > 0$, T is an S_k -tree if it can be built from two S_{k-1} -trees by making the root of one the root of the S_k -tree and making the root of the other the child of the root of the first S_{k-1} -tree.
19. Draw an S_k -tree for $k = 0, 1, 2, 3, 4$.
20. Show that an S_k -tree has 2^k vertices and a unique vertex at level k . This vertex at level k is called the **handle**.
- *21. Suppose that T is an S_k -tree with handle v . Show that T can be obtained from disjoint trees $T_0, T_1, \dots, T_{k-1}$, with roots $r_0, r_1, \dots, r_{k-1}$, respectively, where v is not in any of these trees, where T_i is an S_i -tree for $i = 0, 1, \dots, k - 1$, by connecting v to r_0 and r_i to r_{i+1} for $i = 0, 1, \dots, k - 2$.
- The listing of the vertices of an ordered rooted tree in **level order** begins with the root, followed by the vertices at level 1 from left to right, followed by the vertices at level 2 from left to right, and so on.
22. List the vertices of the ordered rooted trees in Figures 3 and 9 of Section 11.3 in level order.
23. Devise an algorithm for listing the vertices of an ordered rooted tree in level order.
- *24. Devise an algorithm for determining if a set of universal addresses can be the addresses of the leaves of a rooted tree.
25. Devise an algorithm for constructing a rooted tree from the universal addresses of its leaves.
- A **cut set** of a graph is a set of edges such that the removal of these edges produces a subgraph with more connected components than in the original graph, but no proper subset of this set of edges has this property.
26. Show that a cut set of a graph must have at least one edge in common with any spanning tree of this graph.
- A **cactus** is a connected graph in which no edge is in more than one simple circuit not passing through any vertex other than its initial vertex more than once or its initial vertex other than at its terminal vertex (where two circuits that contain the same edges are not considered different).

27. Which of these graphs are cacti?



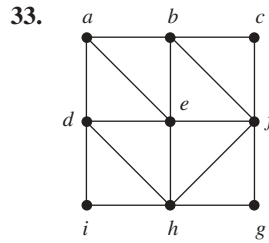
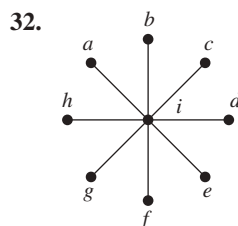
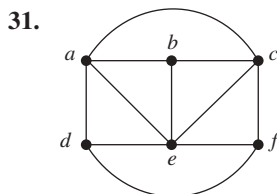
28. Is a tree necessarily a cactus?

29. Show that a cactus is formed if we add a circuit containing new edges beginning and ending at a vertex of a tree.

*30. Show that if every circuit not passing through any vertex other than its initial vertex more than once in a connected graph contains an odd number of edges, then this graph must be a cactus.

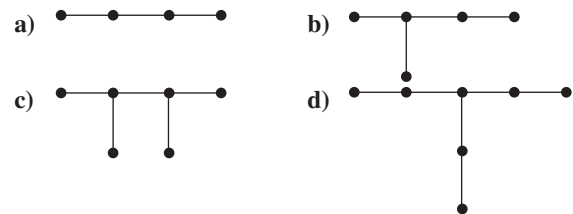
A **degree-constrained spanning tree** of a simple graph G is a spanning tree with the property that the degree of a vertex in this tree cannot exceed some specified bound. Degree-constrained spanning trees are useful in models of transportation systems where the number of roads at an intersection is limited, models of communications networks where the number of links entering a node is limited, and so on.

In Exercises 31–33 find a degree-constrained spanning tree of the given graph where each vertex has degree less than or equal to 3, or show that such a spanning tree does not exist.



34. Show that a degree-constrained spanning tree of a simple graph in which each vertex has degree not exceeding 2 consists of a single Hamilton path in the graph.

35. A tree with n vertices is called **graceful** if its vertices can be labeled with the integers $1, 2, \dots, n$ such that the absolute values of the difference of the labels of adjacent vertices are all different. Show that these trees are graceful.



A **caterpillar** is a tree that contains a simple path such that every vertex not contained in this path is adjacent to a vertex in the path.

36. Which of the graphs in Exercise 35 are caterpillars?

37. How many nonisomorphic caterpillars are there with six vertices?

**38. a) Prove or disprove that all trees whose edges form a single path are graceful.

b) Prove or disprove that all caterpillars are graceful.

39. Suppose that in a long bit string the frequency of occurrence of a 0 bit is 0.9 and the frequency of a 1 bit is 0.1 and bits occur independently.

a) Construct a Huffman code for the four blocks of two bits, 00, 01, 10, and 11. What is the average number of bits required to encode a bit string using this code?

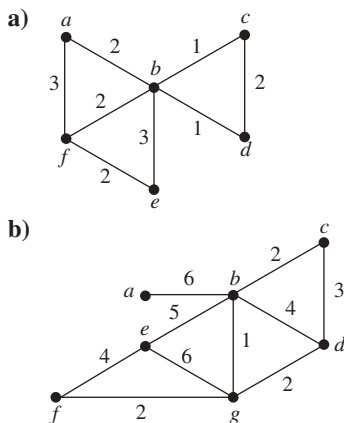
b) Construct a Huffman code for the eight blocks of three bits. What is the average number of bits required to encode a bit string using this code?

40. Suppose that G is a directed graph with no circuits. Describe how depth-first search can be used to carry out a topological sort of the vertices of G .

*41. Suppose that e is an edge in a weighted graph that is incident to a vertex v such that the weight of e does not exceed the weight of any other edge incident to v . Show that there exists a minimum spanning tree containing this edge.

42. Three couples arrive at the bank of a river. Each of the wives is jealous and does not trust her husband when he is with one of the other wives (and perhaps with other people), but not with her. How can six people cross to the other side of the river using a boat that can hold no more than two people so that no husband is alone with a woman other than his wife? Use a graph theory model.

- *43. Show that if no two edges in a weighted graph have the same weight, then the edge with least weight incident to a vertex v is included in every minimum spanning tree.
44. Find a minimum spanning tree of each of these graphs where the degree of each vertex in the spanning tree does not exceed 2.



Let $G = (V, E)$ be a directed graph and let r be a vertex in G . An **arborescence** of G rooted at r is a subgraph $T = (V, F)$ of G such that the underlying undirected graph of T is a spanning

tree of the underlying undirected graph of G and for every vertex $v \in V$ there is a path from r to v in T (with directions taken into account).

45. Show that a subgraph $T = (V, F)$ of the graph $G = (V, E)$ is an arborescence of G rooted at r if and only if T contains r , T has no simple circuits, and for every vertex $v \in V$ other than r , $\deg^-(v) = 1$ in T .
46. Show that a directed graph $G = (V, E)$ has an arborescence rooted at the vertex r if and only if for every vertex $v \in V$, there is a directed path from r to v .
47. In this exercise we will develop an algorithm to find the strong components of a directed graph $G = (V, E)$. Recall that a vertex $w \in V$ is **reachable** from a vertex $v \in V$ if there is a directed path from v to w .
- Explain how to use breadth-first search in the directed graph G to find all the vertices reachable from a vertex $v \in G$.
 - Explain how to use breadth-first search in G^{conv} to find all the vertices from which a vertex $v \in G$ is reachable. (Recall that G^{conv} is the directed graph obtained from G by reversing the direction of all its edges.)
 - Explain how to use parts (a) and (b) to construct an algorithm that finds the strong components of a directed graph G , and explain why your algorithm is correct.

Computer Projects

Write programs with these input and output.

- Given the adjacency matrix of an undirected simple graph, determine whether the graph is a tree.
- Given the adjacency matrix of a rooted tree and a vertex in the tree, find the parent, children, ancestors, descendants, and level of this vertex.
- Given the list of edges of a rooted tree and a vertex in the tree, find the parent, children, ancestors, descendants, and level of this vertex.
- Given a list of items, construct a binary search tree containing these items.
- Given a binary search tree and an item, locate or add this item to the binary search tree.
- Given the ordered list of edges of an ordered rooted tree, find the universal addresses of its vertices.
- Given the ordered list of edges of an ordered rooted tree, list its vertices in preorder, inorder, and postorder.
- Given an arithmetic expression in prefix form, find its value.
- Given an arithmetic expression in postfix form, find its value.
- Given the frequency of symbols, use Huffman coding to find an optimal code for these symbols.
- Given an initial position in the game of nim, determine an optimal strategy for the first player.
- Given the adjacency matrix of a connected undirected simple graph, find a spanning tree for this graph using depth-first search.
- Given the adjacency matrix of a connected undirected simple graph, find a spanning tree for this graph using breadth-first search.
- Given a set of positive integers and a positive integer N , use backtracking to find a subset of these integers that have N as their sum.
- * Given the adjacency matrix of an undirected simple graph, use backtracking to color the graph with three colors, if this is possible.
- * Given a positive integer n , solve the n -queens problem using backtracking.
- Given the list of edges and their weights of a weighted undirected connected graph, use Prim's algorithm to find a minimum spanning tree of this graph.
- Given the list of edges and their weights of a weighted undirected connected graph, use Kruskal's algorithm to find a minimum spanning tree of this graph.

Computations and Explorations

Use a computational program or programs you have written to do these exercises.

1. Display all trees with six vertices.
2. Display a full set of nonisomorphic trees with seven vertices.
- *3. Construct a Huffman code for the symbols with ASCII codes given the frequency of their occurrence in representative input.
4. Compute the number of different spanning trees of K_n for $n = 1, 2, 3, 4, 5, 6$. Conjecture a formula for the number of such spanning trees whenever n is a positive integer.
5. Compare the number of comparisons needed to sort lists of n elements for $n = 100, 1000$, and $10,000$ from the set of positive integers less than $1,000,000$, where the elements are randomly selected positive integers, using the selection sort, the insertion sort, the merge sort, and the quick sort.
6. Compute the number of different ways n queens can be arranged on an $n \times n$ chessboard so that no two queens can attack each other for all positive integers n not exceeding 10.
- *7. Find a minimum spanning tree of the graph that connects the capital cities of the 50 states in the United States to each other where the weight of each edge is the distance between the cities.
8. Draw the complete game tree for a game of checkers on a 4×4 board.

Writing Projects

Respond to these with essays using outside sources.

1. Explain how Cayley used trees to enumerate the number of certain types of hydrocarbons.
2. Explain how trees are used to represent ancestral relations in the study of evolution.
3. Discuss hierarchical cluster trees and how they are used.
4. Define *AVL-trees* (sometimes also known as *height-balanced trees*). Describe how and why AVL-trees are used in a variety of different algorithms.
5. Define *quad trees* and explain how images can be represented using them. Describe how images can be rotated, scaled, and translated by manipulating the corresponding quad tree.
6. Define a *heap* and explain how trees can be turned into heaps. Why are heaps useful in sorting?
7. Describe dynamic algorithms for data compression based on letter frequencies as they change as characters are successively read, such as adaptive Huffman coding.
8. Explain how *alpha-beta pruning* can be used to simplify the computation of the value of a game tree.
9. Describe the techniques used by chess-playing programs such as Deep Blue.
10. Define the type of graph known as a *mesh of trees*. Explain how this graph is used in applications to very large system integration and parallel computing.
11. Discuss the algorithms used in IP multicasting to avoid loops between routers.
12. Describe an algorithm based on depth-first search for finding the articulation points of a graph.
13. Describe an algorithm based on depth-first search to find the strongly connected components of a directed graph.
14. Describe the search techniques used by Googlebot, and other web crawlers and spiders used by different web search engines.
15. Describe an algorithm for finding the minimum spanning tree of a graph such that the maximum degree of any vertex in the spanning tree does not exceed a fixed constant k .
16. Compare and contrast some of the most important sorting algorithms in terms of their complexity and when they are used.
17. Discuss the history and origins of algorithms for constructing minimum spanning trees.
18. Describe algorithms for producing random trees.